CS 334 : COMPILER DESIGN LABORATORY **Scheduled : Feb 10 to 13, 2026**
Premises : Lab 5 [VIA], Lab 1 [VIB] and Lab 7[VIC]

Faculty : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranab, Anup Keshri, Supratim Biswas
Teaching Assistant : Swarna Aishwarya Twinkle (Research Scholar)

Lab Assignment 4 : Generate a scanner and a parser and study their behavior experimentally. All problems use a grammar for declarations.

Objective : 1. Learn how to write regexes and a CFG and automatically generate a scanner and a parser using the tools lex and YACC (Yet Another Compiler Compiler).
2. Requires first manually writing a CFG to generate strings of the language of interest
3. Get familiar with the format of yacc for expressing the grammar rules
4. How to communicate between the parser and its scanner,
5. How to write actions to grammar rules to examine the behavior of the parser,
6. Learn the step by step instructions to generate a scanner and a parser
7. Get familiar with the files produced by yacc and understand their purpose, and finally,
8. To deploy the generated parser to process input strings and interpret the output of the parser.
These skills will get consolidated and strengthened over the next few experiments.

General Instructions :
1. You will need 2 scripts, a lex script (file with extension ".l"), and a yacc script (file with extension ".y"). Lex generates a C program "lex.yy.c" which defines a function yylex(), while yacc generates a C program, "y.tab.c" which defines a function named, yyparse(), which is the parser. Note that yacc generates a special bottom-up parser which we shall discuss in the theory lectures during this and the next week.

- Generation of scanner and parser

```
$ lex lex-script
$ yacc  yacc-script
$ gcc lex.yy.c   y.tab.c  -o parser
```

- Test the parser on sample input string, such as "input.txt" and save the generated output.

```
$ ./ parser   <  input.txt > output.txt
```

P1. You are given 2 files, "declv1.l" and "declv1.y", v1 denotes version 1 design which is done for you. . Read the regexes given in "declv1.l", to simplify the communication between a scanner and parser, single characters have been used. The characters 'i' and 'f' denote "int" and "float" respectively. The letters 'a' and 'b' denote 2 identifiers. The input string, "i a, b ;" is a short form of the declaration statement, "int a, b ;".
The yacc script has a few declarations enclosed between the delimiters, "%{" and "%}", which will be explained during the session. Note the yacc syntax for writing a grammar rule.
1. Nonterminals are followed by a ":", different alternates for same nonterminal are separated by "|" and the end of all rules for the same nonterminal is denoted by ";"
2. C Code may be placed at the end of rule enclosed in "{"and "}".
(a) Generate the scanner and parser as mentioned above, call it say "decl1"
(b) Test your parser on the 3 different input files, "v1inp1.txt", "v1inp2.txt", "v1inp3.txt" and save each output in a separate file. Explain the relation between each input and its output.

Resources Given : "declv1.l", "declv1.y", "v1inp1.txt", "v1inp2.txt", "v1inp3.txt"

P2. The grammar and the regexes are artificial and do not model real inputs. To model actual declarations in C, 2 scripts, "declv2.l" and "declv2.y" are given to you. Read both scripts and observe the differences with respect to that of P1.

(a) Generate a scanner and parser using the following commands.

$ lex declv2.l $ yacc -d declv2.y $ gcc lex.yy.c y.tab.c -o decl2

Examine the contents of the file, "y.tab.h" which plays an important role in setting up communication between a scanner and its parser.

(b) Run "decl2" with the input files, "v2inp1.txt", "v2inp2.txt", "v2inp3.txt", save their outputs and explain the behavior of the parser in each case.

Resources Given : "declv2.l", "declv2.y", "v2inp1.txt", "v2inp2.txt", "v2inp3.txt"

P3. The scripts given to you in P2 will used as base to design better scanners and parsers. Three pairs of input and corresponding output files are given to you in the form : "v3inp1.txt" and "v3out1.txt"; "v3inp2.txt" and "v3out2.txt"; "v3inp3.txt" and "v3out3.txt".

You have to make minimal changes to the scripts of P2 to create 2 new scripts, say, "declv3.l" and "declv3.y". The parser generated by you, say "decl3", should match the desired output files for each of the input files. Freeze the scripts once the requirement is met.

Resources Given : "v3inp1.txt" and "v3out1.txt"; "v3inp2.txt" and "v3out2.txt"; "v3inp3.txt" and "v3out3.txt".

P4. The scripts developed by you in P3 are to be extended to handle one or more C declaration statements. Examine the 3 pairs of input - output files given to you in the form : "v4inp1.txt" and "v4out1.txt"; "v4inp2.txt" and "v4out2.txt"; "v4inp3.txt" and "v4out3.txt". The output files should give you adequate hints to change the scripts so that the behavior of the generated parser matches the expectations.

Resources Given : "v4inp1.txt" and "v4out1.txt"; "v4inp2.txt" and "v4out2.txt"; "v4inp3.txt" and "v4out3.txt"

End of the Experiment 4